Trabalho Prático 05 – Jogo dos Oito (Métodos de Busca)

GCC128–InteligênciaArtificial | Prof.AhmedAliAbdallaEsmin
Aluno: Tobias Maugus Bueno Cougo

1. Implementação

O tabuleiro 3x3 é uma tupla de 9 inteiros (0 = espaço vazio) e o objetivo é **(1, 2, 3, 8, 0, 4, 7, 6, 5)**. Só são gerados movimentos válidos do vazio (Cima, Baixo, Esquerda, Direita), e estados sem solução são detectados pela paridade de inversões. A **Busca em Amplitude (BFS)** usa fila FIFO e um dicionário de visitados (evita loops e guarda o pai de cada estado). O **A*** usa heapq com f(n) = g(n) + h(n), em que g é a profundidade e h é a soma das **Distâncias de Manhattan** das peças até o objetivo; desempates por menor h e ordem de inserção. Nos dois métodos, "nó expandido" é um estado retirado da fronteira (o teste de objetivo ocorre nessa retirada). A interface em **tkinter** permite jogar clicando nas peças adjacentes ao vazio, usar "Novo Jogo", "Embaralhar Estado", "Resolver com BFS" e "Resolver com A*", e navegar pela solução com "Anterior" e "Próximo", vendo método, passos, nós expandidos e tempo.

2. Experimentos

O script experimentos.py sorteia **50 estados iniciais resolvíveis** (seed 42), resolve cada um com BFS e A* e mede nós expandidos e tempo (perf_counter, uma execução por caso). Em todos os casos os dois métodos retornaram soluções de mesmo tamanho (média de 22,1 passos), o que confirma que ambos são ótimos; os testes em testes_corretude.py também reexecutam cada solução e verificam, em amostras, que a heurística nunca superestima o custo real.

Tabela 1. BFS × A* por profundidade da solução (médias por caso).

Profundidade	Casos	Nós BFS	Nós A*	Redução de nós	Tempo BFS (s)	Tempo A* (s)	Speedup
até 16 passos	2	4.252	34	99,2%	0,0084	0,0003	32,8x
17 a 20	12	33.891	188	99,4%	0,0708	0,0014	50,3x
21 a 24	26	102.751	534	99,5%	0,2327	0,0041	57,1x
25 ou mais	10	164.559	1.066	99,4%	0,3896	0,0085	46,1x
Total (50 casos)	**50**	**94.646**	**537**	**99,4%**	**0,2162**	**0,0042**	**52,0x**

3. Análise e Conclusão

Nós expandidos: o A* expandiu em média 537 nós contra 94.646 do BFS — uma **redução de 99,4%** (cerca de 176 vezes menos nós), estável em todas as faixas. O BFS é cego: explora todos os estados até a profundidade da solução e, nos casos mais difíceis, chegou a percorrer 181.261 dos 181.440 estados alcançáveis. A heurística de Manhattan direciona a busca para estados próximos do objetivo, e como é admissível (e consistente), não sacrifica a otimalidade.

Tempo: o speedup médio foi de **52,0x** (mediana por caso: 56,8x), menor que a razão de nós porque cada nó do A* custa mais: calcula a heurística e opera a fila de prioridade, enquanto o BFS só usa uma fila simples. O ganho foi menor nos casos mais rasos (32,8x), em que o BFS ainda é barato. Os tempos vêm de uma execução por caso em Python e dependem da máquina; a redução de nós, por ser determinística, é a medida mais robusta.

Conclusão: para o Jogo dos Oito, o A* com Distância de Manhattan encontra a mesma solução ótima que o BFS expandindo ~0,6% dos nós e em ~2% do tempo. O custo do BFS cresce de forma explosiva com a profundidade, o que mostra o valor de uma boa heurística em problemas de busca com espaço de estados grande.

Link do vídeo de apresentação: https://youtu.be/HJc_S8tLmIU

Link para o repositório GitHub com o projeto completo: https://github.com/TobiasMaugus/IA-Projects